

DMDLPCSHUCI_232

Guia Rapida de Instalación

Table of Contents

1	Requisitos de sistema	2
2	Desarrollador	2
2.1	Despliegue Manual	3
2.2	Creando un Usuario	5
2.3	Generar Inferencia	5
3	Orquestación de servicios	6
3.1	Alta concurrencia	7

1 Requisitos de sistema

Esta guía se encargara de cubrir todos los aspectos esenciales en la instalación del aplicativo tanto para la versión de desarrollador como para el despliegue escalable. Las directrices indicadas y algunos de los comandos están basados en la compatibilidad con Windows, por lo que, para sistemas operativos como Linux y MacOS, los parámetros de instalación varían.

Acuda a la documentación de los comandos de consola en el sistema operativo de lanzamiento y ejecute los comandos equivalentes para efectuar la misma configuración de maquina.

Las características de hardware de su maquina limitaran de manera directa las capacidades del aplicativo. Los requisitos mínimos para este funcionar a un nivel de concurrencia bajo son:

Listing 1: Especificaciones de hardware

```
1 CPU: Dual Core, 2.2 GHz
2 RAM: 4GB
3 HDD: 10GB
4 GPU: ---
```

2 Desarrollador

Replique el código fuente del proyecto, en consola:

Listing 2: Clonar Repositorio Fuente

```
1 gh repo clone DavidPerez99/DMDL_PC_SH_UCI_232
```

Remítase a la rama 'pythonimageJWT' y clone este repositorio en su ambiente local. Si no cuenta con los permisos de administrador para clonar este repositorio via comandos Git, por favor dirijase a: [Repositorio Fuente](#) y remítase a la rama indicada:

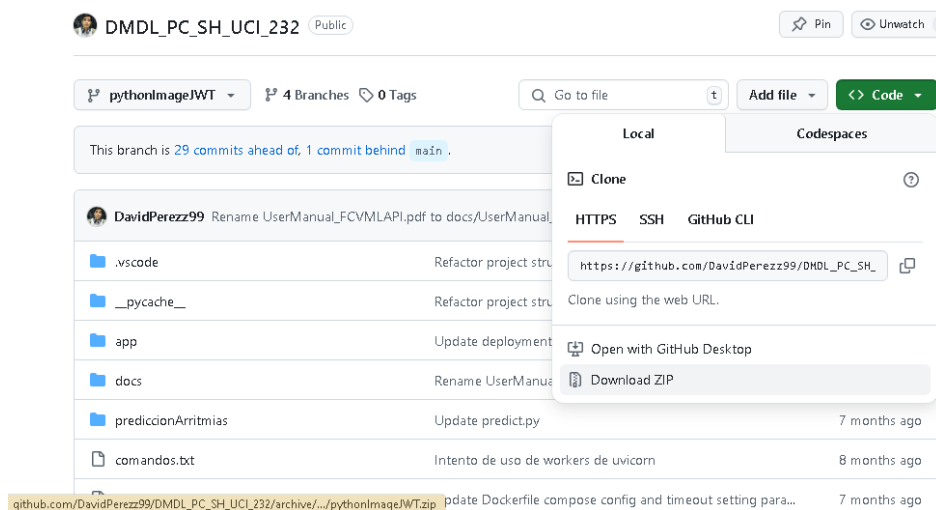


Figure 1: pythonimageJWT repo

Descargue el proyecto como un compresado .zip en 'Download ZIP' o directamente en [.zip](#) y extraiga el contenido en su directorio dedicado al desarrollo y operación.

Una vez hecho esto, el siguiente paso es la definición de los puertos del aplicativo con los cuales estará recibiendo las consultas a sus diferentes servicios. Para esto, mediante el uso de su IDE de preferencia, localice en el directorio 'app', el archivo 'Dockerfile', líneas 15 y 18, establecer el puerto de preferencia:

Listing 3: Definicion de puertos

```

1 # Exponer el puerto en el que correrá la aplicación
2 EXPOSE 5000
3
4 # Comando para correr la aplicación
5 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "5000"]

```

De manera analoga, definir el puerto para el servidor de inteligencia artificial. Para esto ubique el directorio 'prediccionArritmias', archivo 'Dockerfile' y nuevamente, defina su configuración en particular de red para este segundo aplicativo. La aplicación por defecto se ejecutara en los puertos 5000 y 5003 respectivamente.

2.1 Despliegue Manual

El despliegue manual es el idea para ejecutar pruebas y soportar el desarrollo, le permitirá al desarrollador ejecutar pruebas de lógica y resolución de bugs a baja escala sin la necesidad de soportar un despliegue de alta concurrencia. Para esto ultimo ambas aplicaciones 'app1' y 'app2' deben ser puestas en ejecución de forma manual, lo que implica inicializar el servidor de cada una de las apps. Hay diversas formas de inicializar un servidor FastAPI debido a la parametrización de la red y sub red. Para efectos de flexibilidad en el despliegue manual, se recomienda iniciar las apps mediante comandos bash, de la forma:

Listing 4: Lanzamiento Manual por defecto

```

1 uvicorn app.main:app --reload

```

Esto iniciara dentro de la carpeta 'app', el objeto 'main' con la propiedad 'app' en modo recarga, lo que quiere decir que la app estará escuchando a cualquier cambio en su fuente y una vez guardados dichos cambios, estos automáticamente se verán reflejados en la ejecución vigente. Una vez inicializada la app, por defecto esta estara expuesta en el puerto 8000, por lo que si se desea establecer un puerto en la ejecucion manual se debe incluir de la siguiente forma en el comando bash, para este caso en particular, el puerto 5000 para la app1:

Listing 5: Lanzamiento Manual

```

1 uvicorn app.main:app --reload --port 5000

```

Si la inicializacion de la app no tuvo inconvenientes, los logs del inicio de ejecucion deberian verse asi:

Listing 6: Logs de Inicio

```

1 PS C:\Users\artur\OneDrive\Documentos\FCVMLAPI-pythonImageJWT_v1.0\app> uvicorn main:app --reload --port 5000
2 INFO: Will watch for changes in these directories:
3 ['C:\Users\artur\OneDrive\Documentos\FCVMLAPI-pythonImageJWT_v1.0\app']
4 INFO: Uvicorn running on http://127.0.0.1:5000 (Press CTRL+C to quit)
5 INFO: Started reloader process [18232] using WatchFiles
6 2.0.30
7 2.0.30
8 2025-04-03 12:35:41,781 , INFO , Starting DMDL_PC_SH_UCI_232...
9 INFO: Started server process [20292]
10 INFO: Waiting for application startup.
INFO: Application startup complete

```

Para validar si el despliegue fue exitoso, se consulta el endpoint destinado a la informacion de la app mediante el uso de cualquier navegador: <http://127.0.0.1:5000/home>

Listing 7: Logs /home

```

1 2025-04-03 12:46:48,636 , INFO , Starting DMDL_PC_SH_UCI_232...
2 INFO: Started server process [10732]
3 INFO: Waiting for application startup.
4 INFO: Application startup complete.
5 2025-04-03 12:48:19,055 , INFO , {'url': '/home', 'method': 'GET', 'date': '03/04/2025 12:48:19', 'duration':
0.0020182132720947266}

```

En la interaz de navegación se obtendría el siguiente objeto en pantalla:

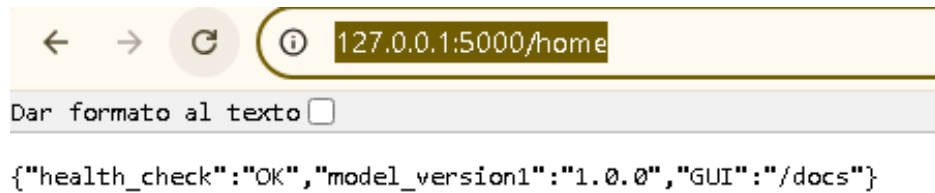


Figure 2: Interfaz de Usuario

Para acceder a la interfaz de pruebas y validar las funcionalidades de la app, dirigirse directamente a `http://127.0.0.1:5000/docs`, este endpoint ejecuta el Swagger de la app, una interfaz de pruebas para interactuar con las diferentes funcionalidades de la app.

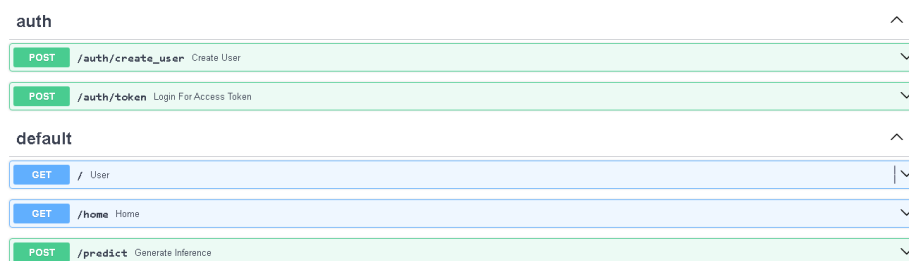


Figure 3: Interfaz de Usuario Swagger

Todas las pruebas que deseen efectuarse en cuanto tengan que interactuar con la aplicación pueden ejecutarse a través de esta interfaz y también se puede apreciar la respuesta de la aplicación ante cualquier entrada de estas.

De la misma forma, la app2 debe ser puesta en ejecución en el puerto 5003. Ubique la app2 dentro del directorio 'prediccionArritmias' el artefacto 'predict.py' y ejecute en consola:

```
uvicorn predict:app2 --reload --port 5003
```

Los logs de la aplicación de inteligencia artificial deberían comportarse de forma similar a los logs del aplicativo app1, no obstante la aplicación dedicada a modelos de inteligencia artificial(app2) no tiene la misma cantidad de servicios que la aplicación app1, al menos para su primera version, el Swagger para esta aplicación se ve así:

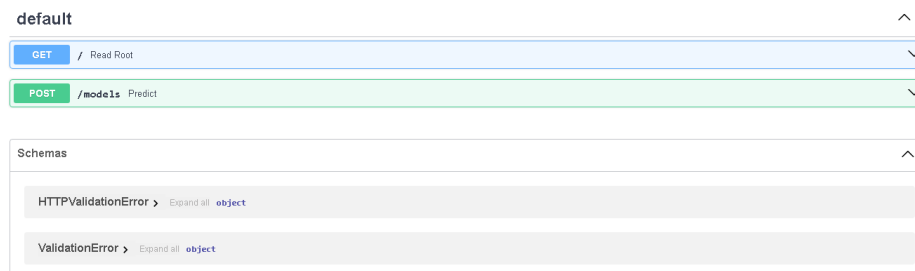
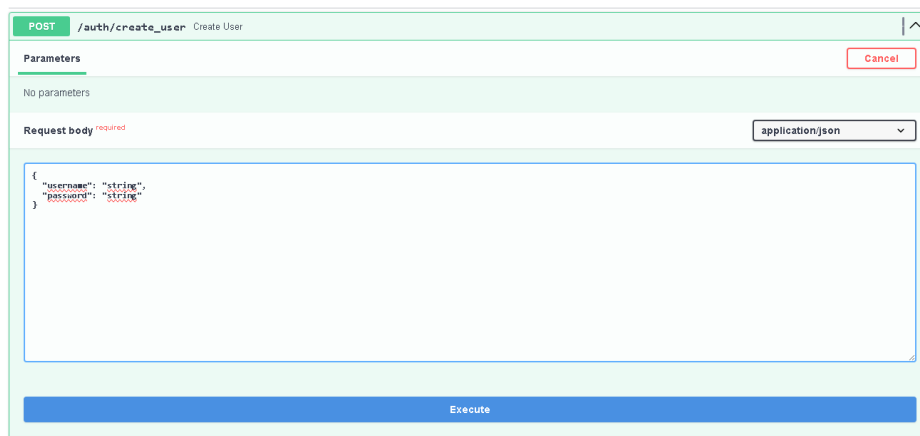


Figure 4: Interfaz de Usuario Swagger app2

Para una ampliación de como acceder a los servicios de estas aplicaciones por favor dirijase al Manual de Usuario capítulos 4 y 5.

2.2 Creando un Usuario

Para poder consumir cualquiera de los servicios que expone esta aplicación deberá contar con un usuario y una sesión, para esto basta con dirigirse a la interfaz de pruebas Swagger y consumir el endpoint `/create_user`, e ingresar los parámetros de nombre de usuario y contraseña:



POST /auth/create_user Create User

Parameters

No parameters

Request body required

application/json

```
{  "username": "string",  "password": "string"}
```

Execute

Figure 5: Asignación de acceso

Allí asignar los valores seleccionados para cada parámetro, para un caso demostrativo username: db_user, password: pass123. Al ejecutar los valores se debería obtener en consola del servidor:

```
2025-04-03 17:56:44,469 , INFO , {'url': '/auth/create_user', 'method': 'POST', 'date': '03/04/2025 17:56:44', 'duration': 0.0069982051849365}
INFO: 127.0.0.1:54166 - "POST /auth/create_user HTTP/1.1" 201 Created
```

Figure 6: Log creación de usuario

Para extender esta explicación, por favor diríjase al Manual de Usuario capítulo 4 sección 4.6. Será necesario que genere su primera sesión para que pueda efectuar pruebas e interactuar con las demás funcionalidades del aplicativo, para esto por favor, lea la guía de autenticación en el Manual de Usuario capítulo 4 sección 4.7.

2.3 Generar Inferencia

Esta aplicación tiene el potencial de alojar en su raíz una múltiple y diversa cantidad de modelos de inteligencia artificial, los cuales pueden ser expuestos a través de servicios o endpoints. Para esta primera versión, solo existe una primera versión de un agente de aprendizaje profundo alojado en `/models` en el puerto 5003, una vez contando con una sesión activa, es posible hacer consultas a esta ruta cumpliendo el formato de consulta por supuesto, que para este modelo de aprendizaje profundo en específico tiene una forma ya establecida, requiriendo cierto tipo y cierta cantidad de información para realizar la inferencia.

Remítase a el Manual de Usuario capítulo 5 sección 5.1.1, encontrara un formato de prueba para consultar en este endpoint y también el formato de los datos que debe llevar la consulta.

3 Orquestación de servicios

Previamente en la sección 2 se apunto el uso de Docker files para la definición de los puertos de funcionamiento de las aplicaciones junto con otros parámetros de ejecución. Esta sección contiene las instrucciones necesarias para lanzar el la aplicación como encapsulado, y para ello primeramente hay que garantizar que el motor de Docker se encuentra instalado en la maquina actual. Cumpliendo con este requisito, desde consola el siguiente paso seria generar el encapsulado como una imagen docker, para esto debemos ubicar una terminal en el directorio donde se aloja la aplicación que queremos encapsular:

Listing 8: Create docker image

```
1 docker build -t <image-name> .
```

En consola debería registrar:

```
PS C:\Users\artur\OneDrive\Documentos\FCVMLAPI-pythonImageJWT_v1.0\app> docker build -t dmdlpcshuci_img .
[+] Building 88.0s (7/9)
=> sha256:9eab895eb84c23106b53e9b3eaa76d790aa11ad723bc8d8e429785e790e5630d 249B / 249B 4.1s
=> extracting sha256:6e909acdb790c5a1989d9cfc795fda5a246ad6664bb27b5c688e2b734b2c5fad 6.7s
=> extracting sha256:245f048c9f8a2432538751286248ab017d49f67a6bc8b78db1a05a20c562ec6f 0.6s
=> extracting sha256:b4a6013a75d147904664f2f911837e64b9e25586c4085e3fdba176719c7f5687 2.7s
=> extracting sha256:9eab895eb84c23106b53e9b3eaa76d790aa11ad723bc8d8e429785e790e5630d 0.8s
=> [internal] load build context 0.8s
=> transferring context: 993.34kB 0.6s
=> [2/5] WORKDIR /app 11.6s
```

Figure 7: Imagen del encapsulado

Una imagen del encapsulado sera creada y para verificar su existencia, ejecutar:

Listing 9: Listar imagenes

```
1 docker image list
```

En consola se despliegan las imagenes disponibles:

Listing 10: Listado log

```
1 PS C:\Users\artur\OneDrive\Documentos\FCVMLAPI-pythonImageJWT_v1.0\app> docker image list
2 REPOSITORY TAG IMAGE ID CREATED SIZE
3 dmdlpcshuci_img latest ab03ddb66331 4 minutes ago 674MB
```

El siguiente paso estaría enfocado en lanzar el contenedor con esa imagen mediante la serie de comandos, en consola:

```
1 PS C:\Users\artur\OneDrive\Documentos\FCVMLAPI-pythonImageJWT_v1.0\app> docker run -d -p 5000:5000 --name
2 dmdlpcshuci_cont dmdlpcshuci_img
3
4 48627d992053357ad5ffca9ee2f73c6b0c92edebd857eaaed6f6c3a3c609aec3
```

Después de crear el contenedor, lo que queda es lanzarlo y validar que su ejecución sea exitosa, para ello, observe el registro en consola:

```
PS C:\Users\artur\OneDrive\Documentos\FCVMLAPI-pythonImageJWT_v1.0\app> docker start dmdlpcshuci_cont
dmdlpcshuci_cont
PS C:\Users\artur\OneDrive\Documentos\FCVMLAPI-pythonImageJWT_v1.0\app> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAME
48627d992053 dmdlpcshuci_img "uvicorn main:app --..." 6 minutes ago Up 11 seconds 0.0.0.0:5000->5000/tcp dmdlpcshuci_cont
```

Figure 8: Aplicación Ejecutada

De forma análoga se debe crear una imagen para la app2 y también un contenedor, luego también se deberá lanzar en un puerto diferente y deberá comunicarse sin problemas como en la versión manual. Para mayor información sobre esta configuración, remitase a el Manual de Usuario.

3.1 Alta concurrencia

Para que la aplicación tenga una manera de escalar ante una abrupta creciente demanda en sus servicios de inferencia, se configuro el despliegue con orquestación swarm, el cual replicara las veces que le sea indicadas la app2 para responder acorde a el numero elevado de solicitudes.

La inicializacion con el orquestador Swarm depende de la definición de un objeto .yaml, para mayor informacion sobre los aspectos a considerar para la configuracion de este objeto, referirse a el Manual de Usuario Capitulo 5.